Software Project Management – IS212

Agile Estimation & Scrum Meetings

Announcements

Announcements – Project Groups & Instructions

- **Project Groups:** please register your teams (5–6 pax) in eLearn today
- The **Project Instructions** will be released Week 4 Monday at 8am
- This will clarify the scope for the "first release" (i.e., Week 12) as well as how we would like you to document your scrum process
  - *Please read it carefully and let us know if you have any questions*
  - *Take note of the advice on **when** to start and finish sprinting* 😊
- **Question:** when the scope of the first release is clarified, what should you do with your existing product backlog?

Recap

Week 2 Recap – User Stories

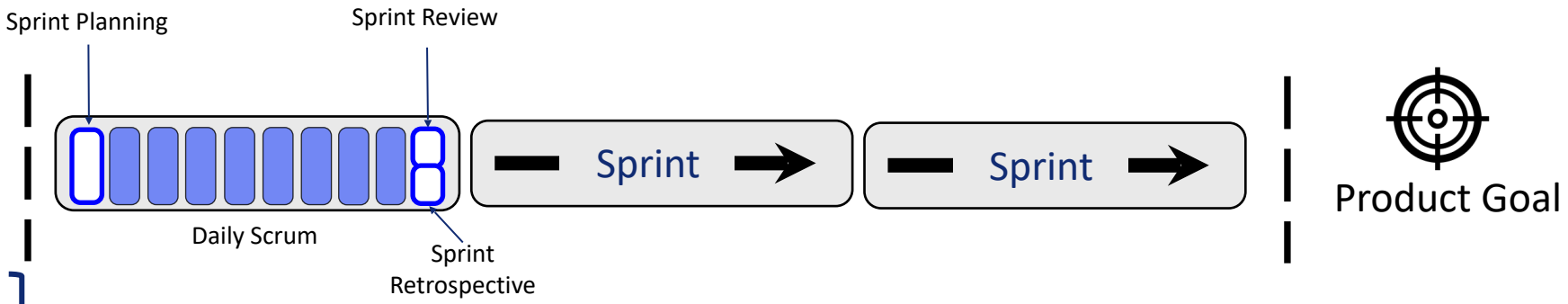
- **User story –** simple feature description from the user's perspective
  - *As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome>*
  - *What the benefits of describing features as user stories?*

- **User story card –** title, user story, and acceptance criteria
  - *What are acceptance criteria? Why are they important?*

- User story (cards) should be INVEST-able; product backlogs DEEP
  - *What do these mnemonics mean? Why are they important?*

- How do epics and story mapping help to organise stories?

Week 2 Recap – User Stories in Scrum



Product Backlog Items

- User Story 1
- User Story 2
- User Story 3
- User Story 4
- User Story 5
- Epic 1

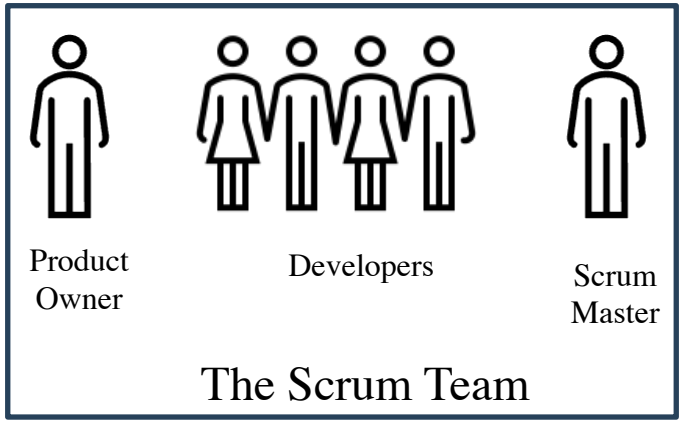
① User Stories can be added, removed or changed

Sprint Backlog

Sprint Goal: To complete …

	To Do	In Progress	Done
User Story 1	Write the front-end code; Write the back-end code; Do the functional testing; Update System Docs		
User Story 2	Task; Task; Task		

② Except in a sprint, where their scope should be **fixed** for the duration



More on sprint planning today 😊

SMU Classification: Restricted

Estimation

Estimation & Scrum Meetings

Objectives

On completing this class, you will be able to:

- Explain the rationale for relative estimation
- Estimate a product backlog using story points
- Plan a sprint according to your team's velocity
- Know how to conduct scrum ceremonies



Outline

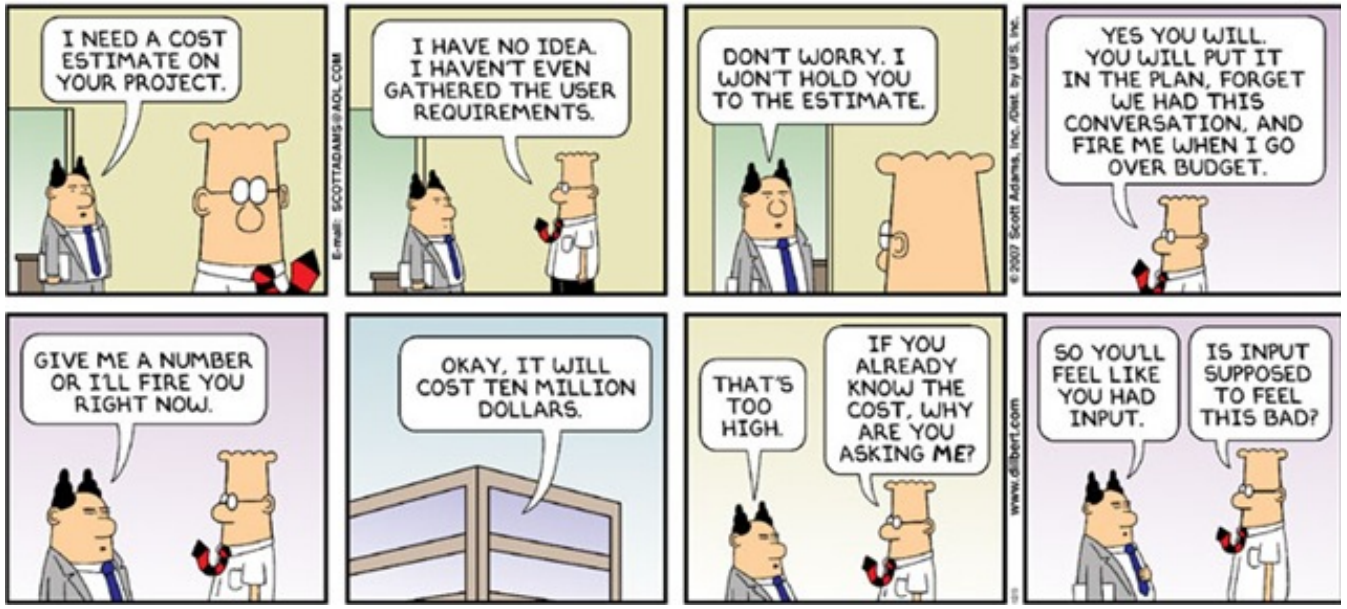
- Estimates vs. Estimation
- Story points, velocity
- Planning poker, burndown charts
- Sprint planning, daily scrum, sprint retro, sprint review

What is an **“estimate”**?

What is **“estimation”**?

Estimation in Software Development

- Software estimates involve:
  - *Calculations, judgement, appraisal*
- …and result in promises based on:
  - *Size, effort, time*
- Estimation is the process by which estimates are arrived at



basis for competitive contracts *(money, time)*

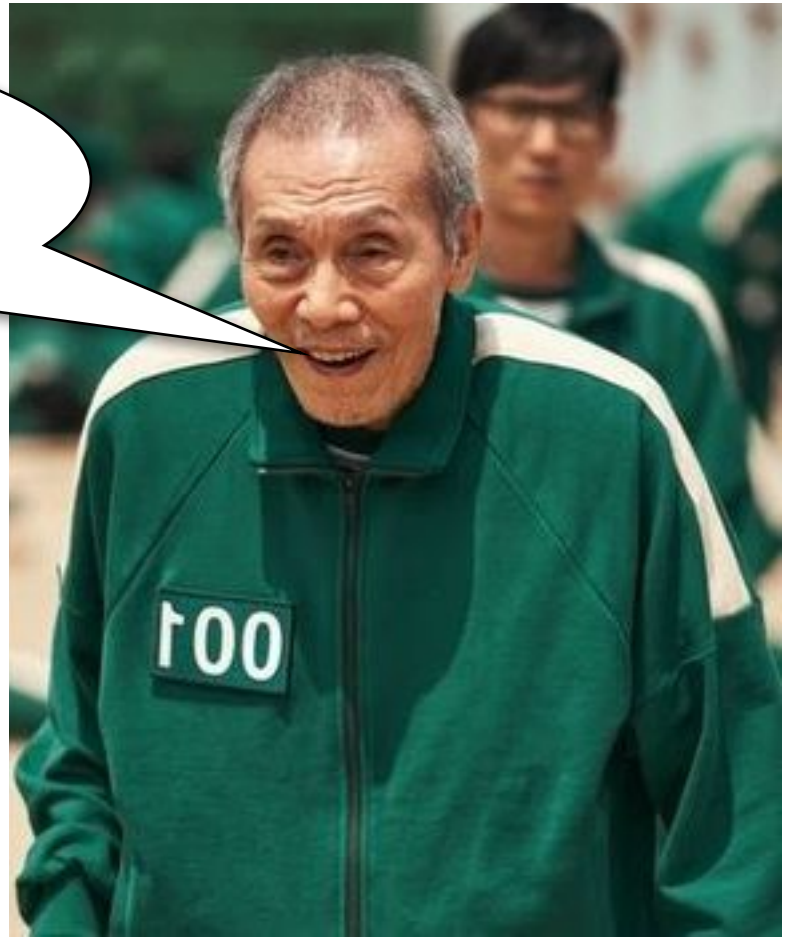
efficient resource planning

Why are (good) software estimates **important**?

determine scope/timing of releases

basis for comparing “actual” vs. “planned” work done

"This project requires 50 person-months of effort"



In most **single-point estimates**, the level of certainty is not "100%"



…50 person-months later

Why is Estimation Difficult in Software Development?

Inaccurate information about a project



Inaccuracies in the estimation process



Difficulties with making good estimation



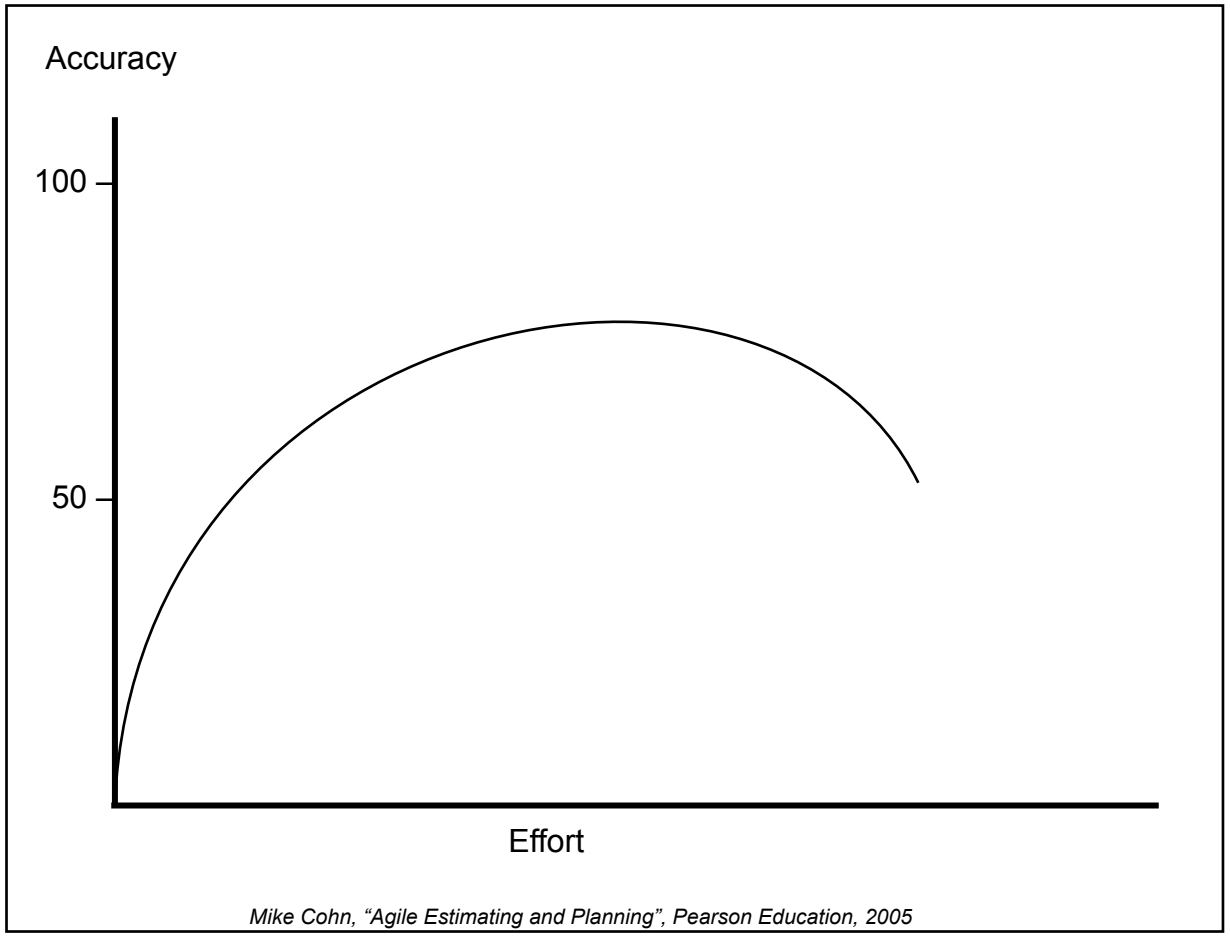
Inaccurate information about organization



Oscillating requirements

Based on: Subhajit Datta, "Software Engineering: Concepts and Applications", Oxford University Press, 2010

More Effort in Estimation ≠ Better Estimates



Mike Cohn, "Agile Estimating and Planning", Pearson Education, 2005

*Beyond a certain point, more effort invested in estimation will **not** lead to more accurate estimates!*

Why is this so?

What Leads to Better Estimates?

- Clarity of the goal
- Clarity of the steps towards the goal
- Clarity of duration of each step. **How?**
  - *Experience with similar projects in the past.*
  - *Experience with required technologies and tools.*
  - *Experience working with the same teams and customers.*
  - *Knowledge of existing dependencies across teams or projects.*
- Anticipation of a change and accommodating the unknown

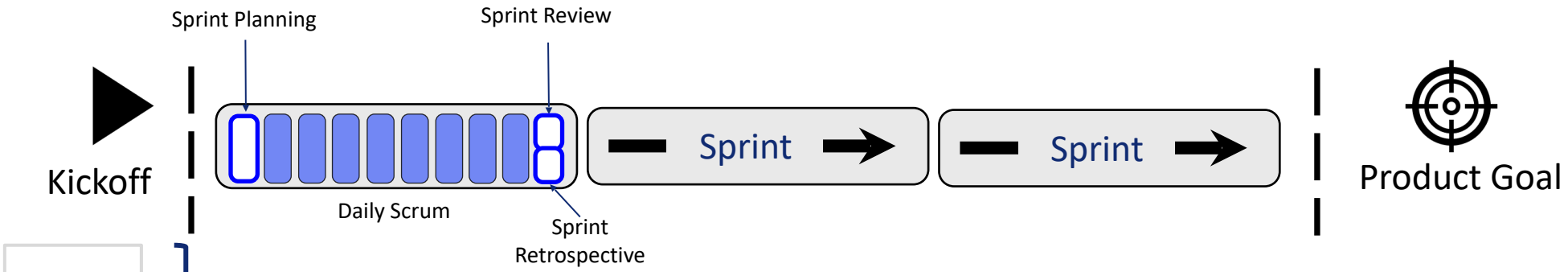
"[…] estimating and planning processes must themselves be agile.
Without agile estimating and planning, we cannot have agile projects."

Mike Cohn

Agile Estimation



Estimation in Scrum



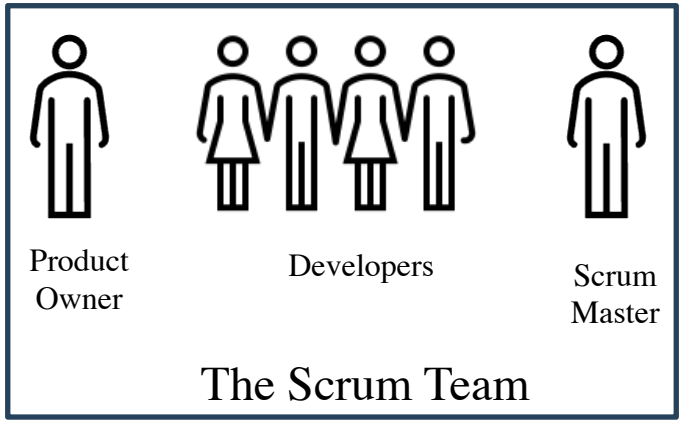
Product Backlog Items

- User Story 1
- User Story 2
- User Story 3
- User Story 4
- User Story 5
- Epic 1
- Epic 2
- Epic 3

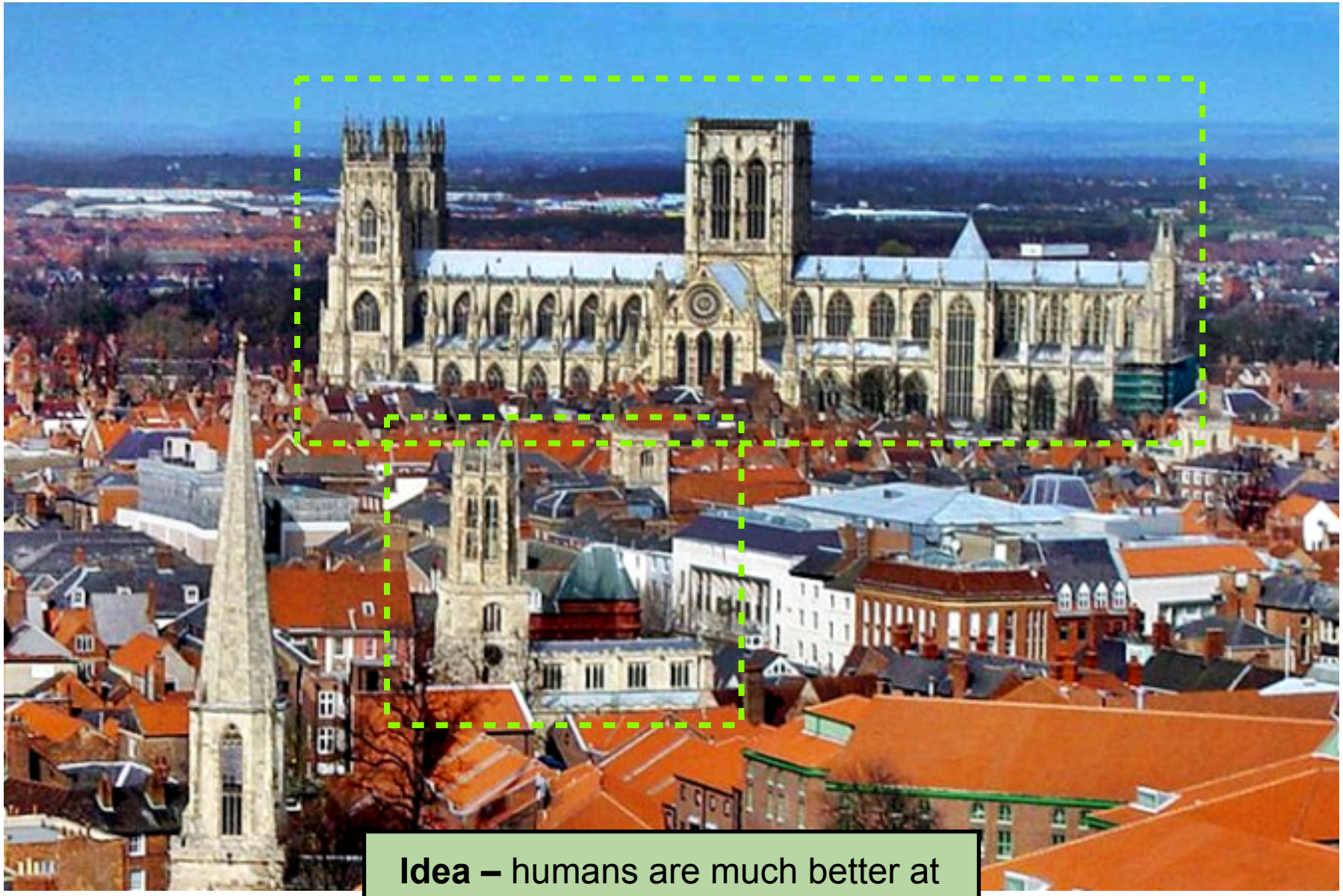
Sprint Backlog

Sprint Goal: To complete …

	To Do	In Progress	Done
User Story 1	Write the front-end code; Write the back-end code; Do the functional testing; Update System Docs		
User Story 2	Task; Task; Task		



What needs to be estimated in scrum?



Idea – humans are much better at making **relative estimates**

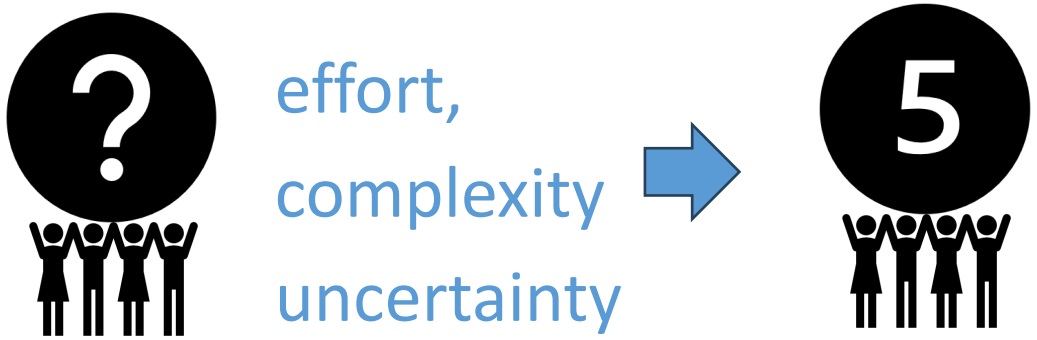
Agile Estimates – Story Points

- Story points are a unit of measurement used to estimate the effort required to complete a user story
- They are **NOT** an absolute measurement like the numbers of hours
- Rather, story points are based on **relative estimation**. E.g.:
  - Items with 2 story points require "roughly" **2x more effort** than items with 1 story point
  - Items with 1 story points require "roughly" **one third the effort** of items with 3 story points
- In practice, estimation is a bucketing exercise and there isn't a precise mathematical relationship between point values
  - Eventually, teams build an intuition for what should be bucketed as a 1-pointer, 3-pointer, 5-pointer, etc., based on the effort required relative to **previously completed** stories

What is the Meaning of a Story Point?

- Story points have no set meaning
  - There is no set formula for defining the point value of a story
  - They are an amalgamation of the amount of effort involved in developing a feature, the complexity of developing it, the inherent risk, etc.

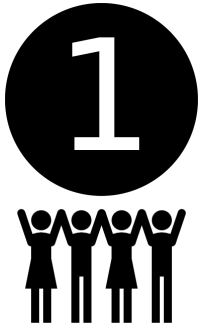
- What matters is that the team takes a consistent and consensus-based approach to estimating stories relative to estimated stories



Team decision but usally developers

How to Start?

- **Option #1:**
  - Select a user story that you expect to require the least effort
  - Assign it an estimate of 1 or 2 story points
  - Don't fret 😎



- **Option #2:**
  - Select a user story that seems somewhat medium in effort
  - Assign it an estimate in the middle of the expected range, e.g. 3 or 5 points
  - Don't fret 😎

- That's it! Estimate the next story relative to this one

Story Point Values

- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, … ? ❌
  - Too fine-grained
  - Larger stories have more uncertainties – harder to estimate accurately
  - Waste of time to argue whether a story is 14 pts vs. 15 pts (etc.)
  - We rather want to put the story in the 'right ballpark' and move on

- Studies show that humans perceive differences between smaller numbers more precisely than differences between larger numbers
  - E.g. the difference between 1 vs. 3pts is easier to perceive than, say, 15 vs. 16pts

- Consequently, many scrum teams estimate using **only** the values of a non-linear scale, e.g. modified fibonacci:
  - 1, 2, 3, 5, 8, 13, 20, 40, 100

Why the big jumps?

Break down anything 20 or bigger. User story is too big if it cant be estimated

"smallest" story

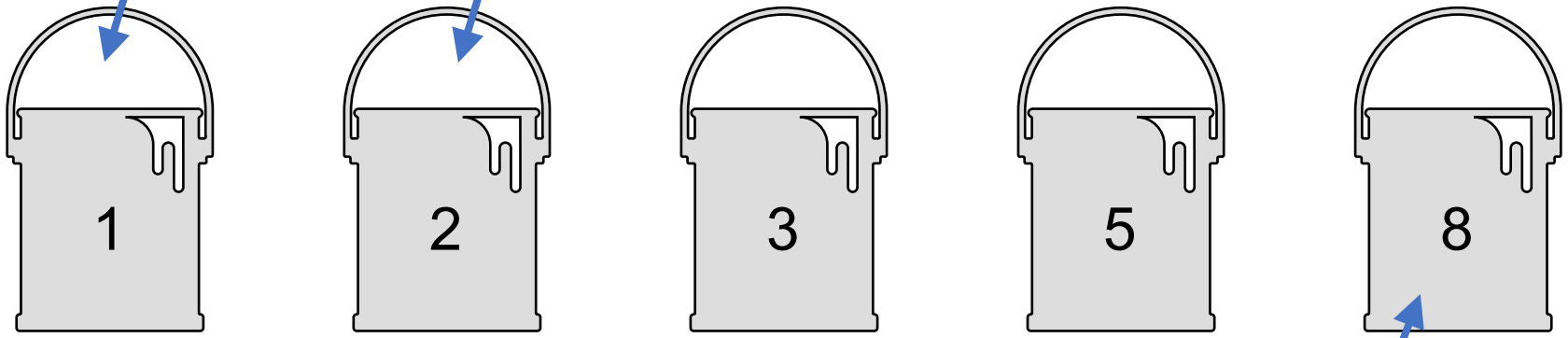




roughly double the effort of your baseline 1 point story

assign 1 point

assign 2 points



assign 8 points

What if a story is larger than 8 points?

It likely needs more conversation (and perhaps) splitting



more than 5 but less than 8 …?

Story points: …

User Story

*As a **student**, I want to **view my profile** so that I can **ascertain that my data is correct***

Acceptance Criteria

Ensure the student can:

* *View their name, email address, year of study, and campus ID*

Story points: …

User Story

*As a **student** I want to **view an overview of forthcoming courses** so that I can find out **which ones I am eligible for***

Acceptance Criteria

Ensure the student can:

* *View course titles, codes, descriptions, and pre-requisites*
* *View only courses that they haven't taken yet*
* *Filter by courses they are eligible for*

Story points: …

User Story

*As a **faculty**, I want to **update the pre-requisites** of my course so that students can **only enrol once they have the required skills***

Acceptance Criteria

Ensure the faculty can:

* *Select a course that they manage (only)*
* *Select only from courses that they manage*
* *View the current pre-reqs for the course*
* *Remove a pre-req from the course*
* *Add (different / non-mutex) course(s) as a pre-req*

Story points: …

User Story

*As an **Associate Dean**, I want to **create a course** in the LMS so that **a faculty can start preparing it***

Acceptance Criteria

Ensure the Associate Dean can:

* *Enter a course name and term*
* *Enter a valid distinct course code for the AD's school*
* *Select a faculty as course manager*
* *Create the course*

Story points: …

User Story

*As a **student**, I want to **ask questions in a forum** so that I can **seek help from my peers***

Acceptance Criteria

Ensure the student can:

* *…*

Common Estimation Techniques

- **Expert opinion**
  - The expert relies on their intuition, experience or gut-feel to provide an estimate
- **Analogy**
  - The estimator compares the story being estimated to one or more other stories
  - A story that is twice the size, will be given an estimate twice as large
- **Disaggregation**
  - A user story is split into smaller, easier-to-estimate pieces (divide-and-conquer)
  - Large items are difficult to estimate; there are also very few similar stories to compare

- Alternatively, combine all three techniques using planning poker 

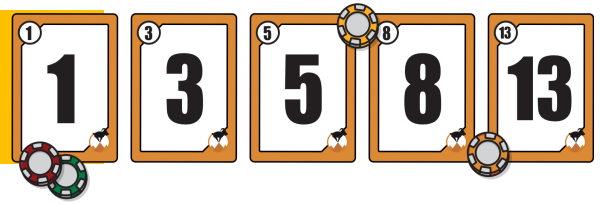


http://www.agilebuddha.com/agile/how-to-do-story-point-sizing-with-planning-poker/

Estimation using planning poker



Planning Poker – Steps



1. Each estimator given a deck of cards: 1, 2, 3, 5, 8, 13, 20, 40, and 100.
2. For each user story to be estimated, a moderator reads the description and answers any questions that the estimators have.
3. Each estimator privately selects a card representing his or her estimate.
   - *Cards are not shown until each estimator has made a selection.*
   - *At that time, all cards are simultaneously turned over and shown so that all participants can see each estimate.*
   - *The Product Owner should **not** be an estimator (but can be a moderator)*
4. If estimates differ, the high and low estimators explain their estimates and discuss them among the team.
5. After the discussion, each estimator re-estimates by selecting a card.
6. Steps 3 - 5 are repeated until the estimates converge.

https://www.scrumpoker-online.org/ **https://scrumpoker.online/**

SMU Classification: Restricted

Velocity & Burndown Charts

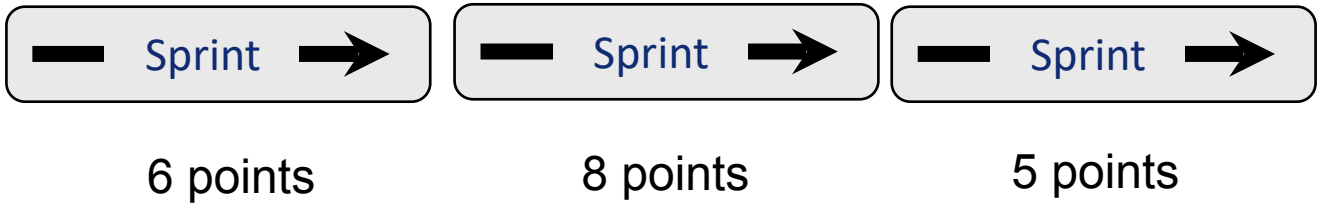
Velocity – *how much work can we complete*

- Velocity is a measure of a team's rate of progress



- Measured by summing the no. points of the user stories fully completed in the last sprint
- Example – in two sprints the team completed…
  - ***Sprint 1:*** *Three 5-point stories* ***<— velocity 15***
  - ***Sprint 2:*** *Three 4-pt stories, three 1-pt stories* ***<— velocity 15***
- Thus, our best guess is that this team can manage 15-pts worth of user stories in the next sprint *(points* ***not*** *number of stories!)*

Measuring Velocity

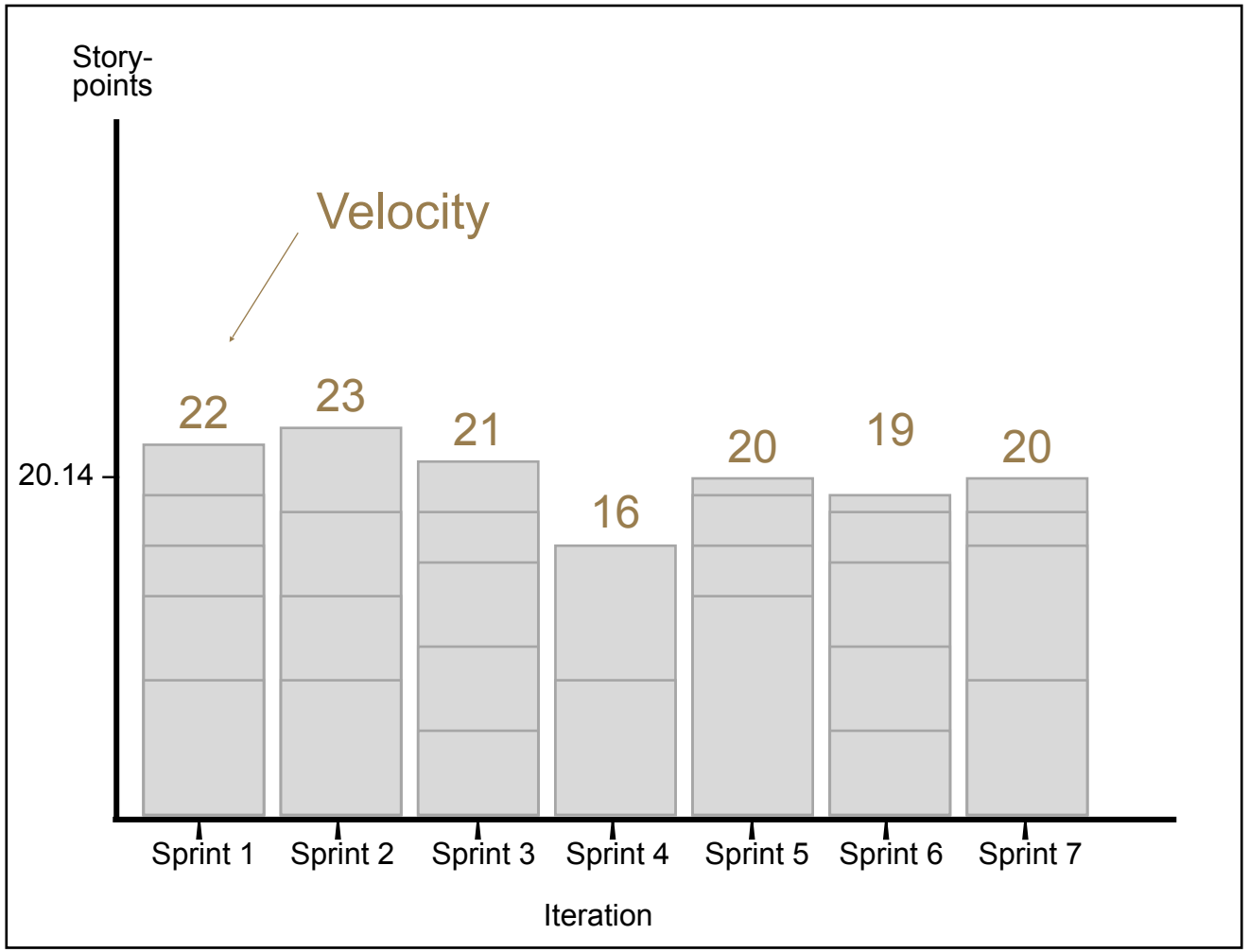


- **Average velocity** for the team : (6 + 8 + 5) / 3 = 6

—> *reasonable to plan 6 story points of work in the next sprint*

Will velocity change throughout the project? Why?

Measuring Velocity



Velocity lets teams understand how many story points they can reasonably tackle in the next sprint.

For the first few sprints, teams have to make assumptions based their past work and experience.

Deriving the Time Needed to Complete a Release / Project

Can you derive (i.e. calculate) how long it will take to complete this project?

- Suppose all user stories in the product backlog are estimated, and the **total sum** of these estimates is 200 story points
  - *This is the estimated size of the system*
- Suppose that we know from experience that the team can on average complete 20 story points in each two-week iteration

Estimated total size: **200**

Team's velocity: **20**

Duration of iteration: **2 weeks**

Deriving the Time Needed to Complete a Release / Project

Can you derive (i.e. calculate) how long it will take to complete this project?

Fixed scope

How many **iterations** will it take to deliver all story points?
*(200 story points) / (20 story point per sprint) = **10 sprints***

How many **weeks** will it take to deliver all story points?
*(10 sprints) * (2 weeks per sprint) = **20 weeks***

Fixed time

How many story points can the team deliver in case?
Does the customer decide to stop development after 15 weeks?
*floor ((15 weeks) / (2 weeks per sprint)) * (20 story points per sprint) = **140 story points***

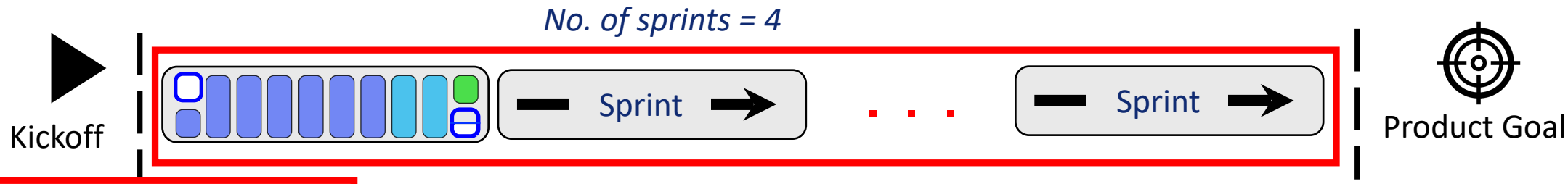
Estimated total size: **200**

Team's velocity: **20**

Duration of iteration: **2 weeks**

Fixed time so take the floor for the number of sprints

Another Example



What is the effort required? 22 story points

User Story 1	3
User Story 2	3
User Story 3	5
User Story 4	5
User Story 5	2
User Story 6	1
User Story 7	2
User Story 8	1

Total: 22 story points

Velocity: 7

No. of sprints = 22 / 7
= 3.143
= 4 (round up)

Duration of each sprint: 2 weeks

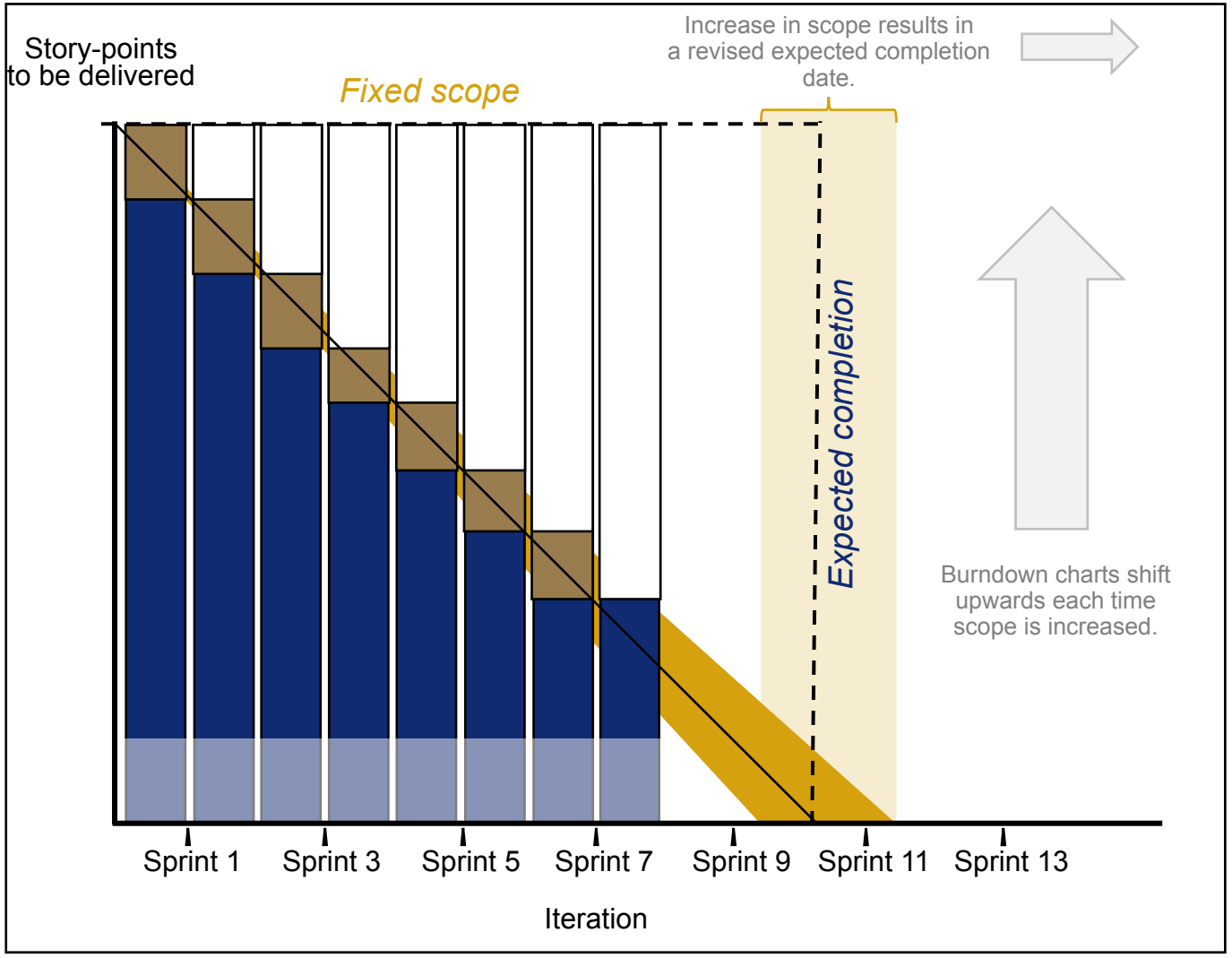
Total no. of weeks: 2 * 4 = 8

(NB: may want to include a velocity 'buffer', e.g. 10%, reserved for unexpected bugs.)

Separation of Effort and Duration

- We estimate the **effort** of user stories (using a relative approach)
- We measure velocity, and use it to derive duration
  - i.e. we are NOT directly estimating duration
- If a team's velocity changes, we can re-calculate the completion of the release / project without re-estimating any user stories
- **Example:**
  - Suppose a team estimates a release to include **200** story points of work
  - This initially leads them to believe that they can complete it in **10** sprints, if they work with their usual velocity of **20** story points per sprint
  - Once the project begins, they observe velocity of **16** story points. Without re-estimating any work, they can now correctly identify that the project will take **13** sprints (rather than **10**)

Burndown Charts – *tracking the work remaining*



Burndown charts *show how the cumulative story points delivered reduce the* ***remaining effort*** *to deliver the project.*

They help track the completion date while delivering a fixed scope project.

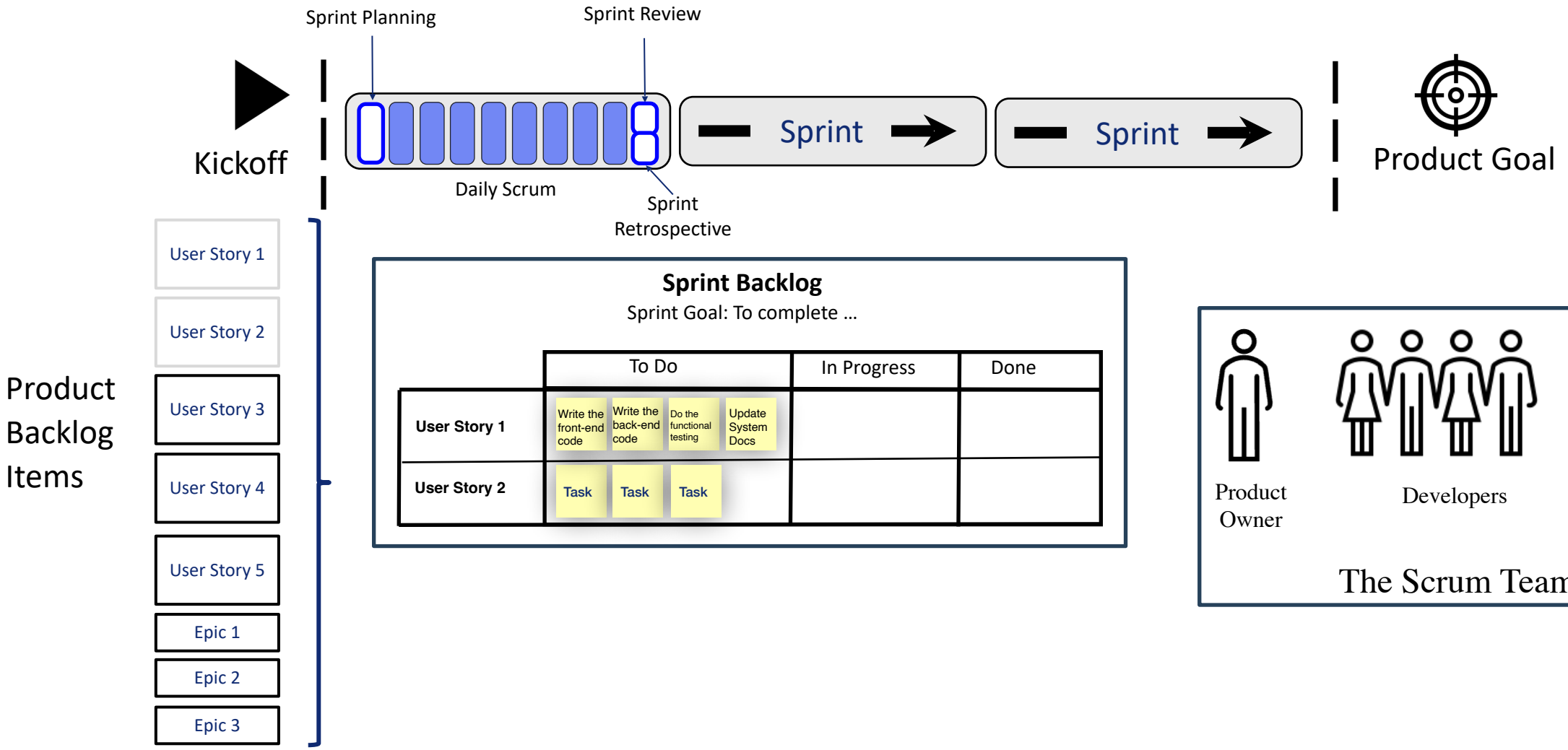
Used to see whether everything is going according to schedule

Responsibility of the product owner to see and manage progress

SMU Classification: Restricted

Scrum Meetings

Recap – Overview of Scrum



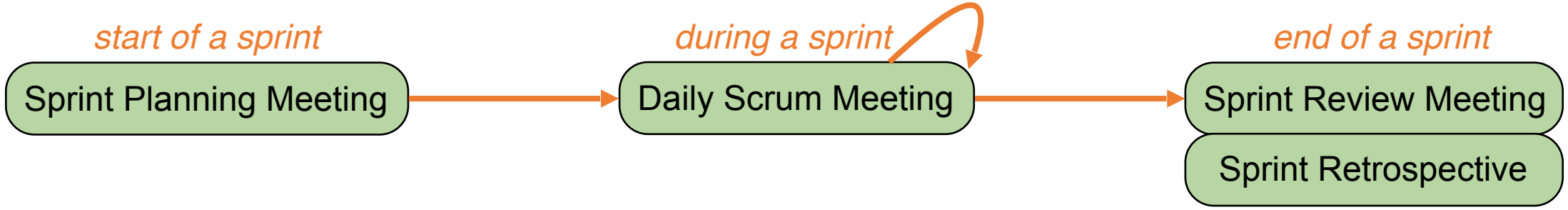
Recap – Overview of Scrum

- Sprints are fixed-length (e.g. 2, 3, or 4 week) iterations that aim to achieve an agreed sprint goal, e.g. a particular software increment
  - For the IS212 project, we recommend choosing a sprint length of **1 or 2 weeks**
  - Sprint goal is frozen for the length of a sprint

- Sprints are lightly managed using "time-boxed" scrum meetings (or 'ceremonies')

Why is time-boxing important?

Time boxxed means finish or not once the time is up the meeting ends



Sprint Planning

- Sprint planning takes place at the start of a sprint, time-boxed to 2hrs * no. of weeks in the sprint
  - E.g. 4hrs for a 2 week sprint

- **Purpose:**
  - Establish the overall **sprint goal**
  - Identify the product backlog items that should be completed in the sprint
  - Align expectations among the team

- Attended by the whole scrum team



https://www.scrum.org/resources/what-is-sprint-planning

Sprint Planning – Activities

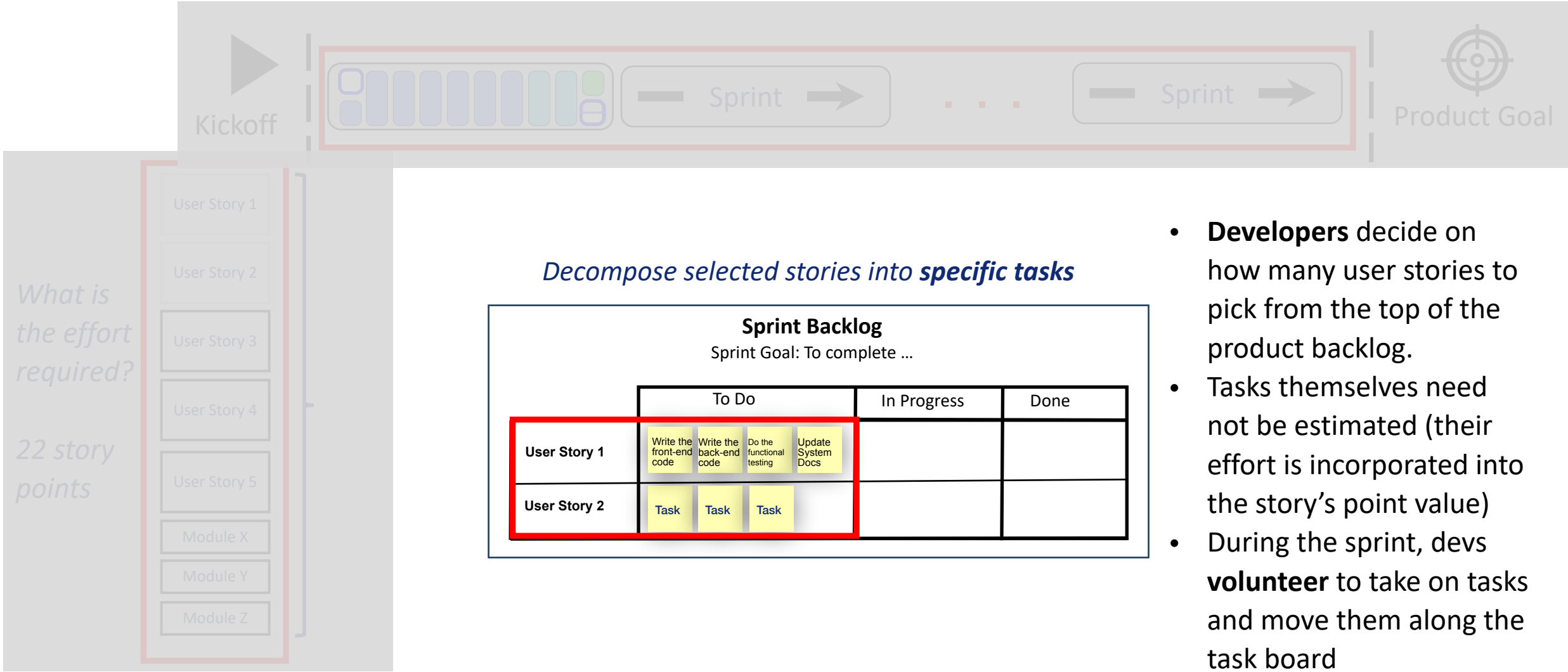
- ***Product Backlog Review*** **–** Product Owner presents the highest-priority items, answering questions about scope and requirements

- ***Estimation*** **–** planning poker to (re-)estimate the backlog items; identify stories that need further clarification or splitting



- ***Sprint Backlog*** **–** based on the team's velocity, commit to some stories that together contribute to a coherent **sprint goal**
  - Place the stories on a scrum task board
  - Start decomposing those stories into **specific tasks**

Sprint Planning – Tasks



*Decompose selected stories into **specific tasks***

Sprint Backlog

Sprint Goal: To complete …

	To Do	In Progress	Done
User Story 1	Write the front-end code; Write the back-end code; Do the functional testing; Update System Docs		
User Story 2	Task; Task; Task		

- **Developers** decide on how many user stories to pick from the top of the product backlog.
- Tasks themselves need not be estimated (their effort is incorporated into the story's point value)
- During the sprint, devs **volunteer** to take on tasks and move them along the task board

Sprint Planning Benefits

- Aligns the team on what needs to be achieved in the next sprint
- Avoids burnout – the team determines an achievable goal for the sprint and selects work that matches their velocity
- Enhanced communication – opportunity to discuss priorities, technical challenges, and tasks

Sprint Planning Challenges

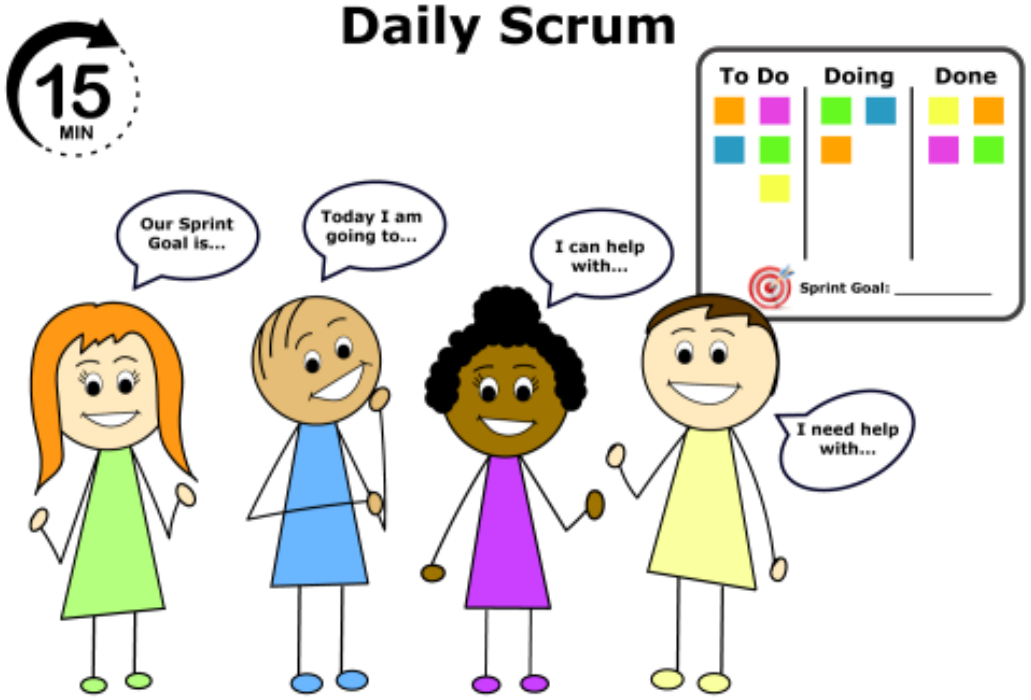
- Overcommitment – especially when the team's velocity is not yet known
- Scope creep – when trying to fit too much work into a single sprint
- "Analysis Paralysis" – spending more time planning than actual development work (must time-box!)
- Team dynamics – differences in communication styles, experience, or conflicts can hinder productive discussions and decision-making

Daily Stand-up (aka Daily Scrum)

- The daily stand-up takes place each day, time-boxed to 15 mins

- **Purpose:**
  - Help the team to synchronise
  - Provide status updates
  - Identify obstacles (“blockers”)

- Attended by the whole team; facilitated by the scrum master



https://www.scrum.org/resources/what-is-a-daily-scrum

Daily Stand-up (aka Daily Scrum)

- There are many different ways to conduct a stand-up
  - Many different tools (e.g. Miro), bots, etc. to support teams too

- Popular approach is for each team member to answer:
  - What did I do yesterday?
  - What will I do today?
  - Are there any impediments in my way?

What kind of impediments might be encountered?

- **Important:** this is to synchronise and assist, not to blame!

Stand-up Benefits

- Enhances communication and collaboration
- Reduces the feeling of isolation (especially in the age of WFH)
- Keeps the team focused and aligned on goals
- Quickly identifies and resolves blockers
- Helpful beyond scrum – many waterfall teams do stand-ups!

Stand-up Challenges

- Meeting can become dominated by certain team members / personalities
- Lack of engagement / silence from certain team members
- Meetings turning into problem-solving sessions
- Scrum master must carefully facilitate and time-box the session to mitigate the above!

Sprint Review

- The sprint review takes place at the end of a sprint, time-boxed to 1hr * no. of weeks in the sprint
  - E.g. 2hrs for a 2 week sprint

- **Purpose:**
  - Demonstrate the software increment (i.e. the work that has been 'done')
  - Gather feedback on the increment
  - Adapt the product backlog as necessary

- Attended by the whole team plus key stakeholders invited by the product owner



https://www.scrum.org/resources/what-is-a-sprint-review

SMU Classification: Restricted

Sprint Review

- **Key Activity:** determining whether or not the sprint goal was met
- Team explains which sprint backlog items have been “Done” and what has not been “Done”

What does “Done” mean?

- Developers demonstrate the “Done” work and answer questions
- Feedback (e.g. required enhancements) is incorporated into the product backlog for future sprints
- User stories that are not “Done” are returned to the product backlog, and do not count towards the sprint’s velocity

Sprint Review Benefits

- A chance to gather immediate feedback from stakeholders on the software increment
- Increase transparency and accountability by demonstrating actual work done in the sprint
- Helps to ensure development work remains aligned with the overall product vision
- Team motivation! Recognise a job well done 😊

Sprint Review Challenges

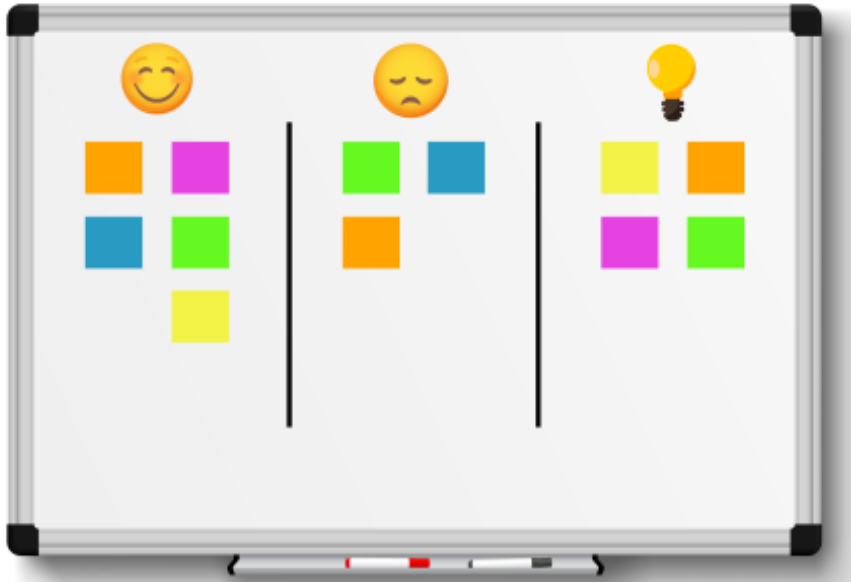
- The meaning of “Done” requires consensus **(see Week 4!)**
- Scope creep – stakeholders may use sprint reviews to push for new features / changes
- Incomplete work can demoralise team members or lead to awkward demonstrations – address via process improvements
- Partially completed stories impact planning for subsequent sprints

Sprint Retrospective

- The sprint retrospective concludes the sprint, time-boxed to 45mins * no. of weeks in the sprint
  - E.g. 1.5hrs for a 2 week sprint

- **Purpose:**
  - Reflect on what went well (and what didn't) during the sprint
  - Identify areas of improvement in processes, tools, collaboration
  - Empower the team with a structured opportunity to voice their opinions, concerns, and suggestions

- Attended by the whole team and is facilitated by the scrum master



https://www.scrum.org/resources/what-is-a-sprint-retrospective

Sprint Retrospective

- The scrum master can use various techniques to elicit feedback on the sprinting process from team members

- For example, using:
  - 4Ls: Liked, Learned, Lacked, Longed For
  - Start-Stop-Continue



- Based on the insights, the team collaboratively identifies specific actions or improvements they want to implement in the next sprint
  - They should be actionable, realistic, and within the team's control

Sprint Retro Benefits

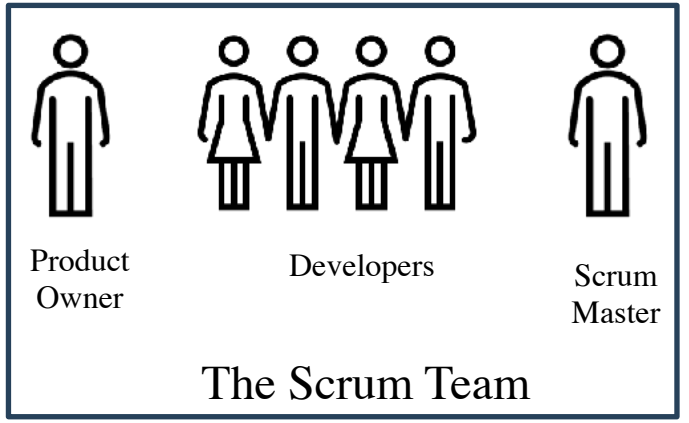
- A structured opportunity to foster continuous improvement
- Encourages a culture of self-reflection and proactive problem-solving
- Empowers team members voice their opinions and contribute to the team's success
- Ultimately (ideally), leads to stronger team cohesion and shared accountability

Sprint Retro Challenges

- Avoiding an overly negative focus – be sure to balance the issues with recognition of successes and achievements!
- Identifying improvement actions is important, but they also need to be followed-through
- Encouraging open and honest communication is difficult – team members may be afraid of judgement or conflicts

Backlog Refinement (aka 'Grooming')

- Finally, backlog refinement is often included as a scrum meeting / ceremony
  - Can occur at any time
  - Does not interfere with the sprint backlog of an ongoing sprint



- **What's it about?**
  - Reflect and prioritise items in the product backlog
  - Ensure that it is ***DEEP***
  - Review user stories, add details, reorder priorities, and split or combine items

- Product owner must be there, but other scrum team members and/or stakeholders may be present too

Scrum FAQs

- **Do all of my sprints need to be the same length?**
  - Typically, yes: it leads to better predictability and 'rhythm'

- **Can the sprint backlog be changed during a sprint?**
  - Typically, no: it negatively impacts predicability. ***However –***
  - Scrum Guide: *"The Sprint Goal is created during the Sprint Planning event and then added to the Sprint Backlog. As the Developers work during the Sprint, they keep the Sprint Goal in mind."*
    "If the work turns out to be different than they expected, they collaborate with the Product Owner to negotiate the scope of the Sprint Backlog within the Sprint without affecting the Sprint Goal."
  - If the work turns out to be significantly different, it may be better to terminate the sprint and plan another with this new knowledge

Scrum FAQs

- **What happens if a user story isn't completed within a sprint?**
  - The story is returned to the product backlog and does not count towards the velocity of the sprint
  - Velocity is intended to reflect the value delivered to the customer, and if the story is not complete, it's not considered as delivering value… yet

- **If a partially complete story returns to the product backlog, should I re-estimate it?**
  - Different practitioners have different views
  - Some may re-estimate it, reflecting that it requires (likely) less effort to complete given that some of the tasks were completed. This is helpful when performing velocity-driven sprint planning
  - Others argue that you shouldn't re-estimate it, as otherwise you don't get full credit for the work when it is completed (and it adds an unappealing step to the burndown chart)
  - The following article argues for the latter approach, emphasising the value of capacity-driven sprint planning instead: **https://www.mountaingoatsoftware.com/blog/should-you-re-estimate-unfinished-stories**

Scrum FAQs

- **Can I have a sprint purely for infrastructure design / set up?**
  - Typically, there is an expectation that every sprint leads to some increment of the software
  - If the sprint goal is ***only*** to set up developer infrastructure, then there is no product increment to demonstrate to stakeholders in the sprint review
  - However, some scrum teams plan "special" sprints to address such matters
    - E.g. "Design sprints", "sprint zero", …
    - Research them and justify if you want to include them in your team's process!

Scrum FAQs

- **How do multiple teams coordinate on a larger system?**
  - One approach is "Scrum of Scrums", where team representatives meet regularly to coordinate and resolve cross-team issues
  - Another is a more "hybrid" approach: overall planning and integration following a waterfall-like sequence (e.g. fixed milestones), while each sub-team uses scrum for iterative development

- **How does a scrum team involve an external vendor?**
  - The vendor can be treated as a separate (scrum) team or stakeholder, with integration points defined through agreed-upon deliverables, APIs, or sprint reviews
  - If close collaboration is needed, a vendor representative may attend relevant scrum meetings (e.g. sprint planning or review)
  - Coordination often works best with clearly defined interfaces and expectations, supported by contracts that allow for iterative delivery.

SMU Classification: Restricted

Exercise & Lab

Exercise & Lab

- **Exercise:**
  - Collate the user stories your group put together last week
  - Start estimating your product backlog using the techniques discussed today
  - 'Groom' and refine it according to the customer's clarifications, and split any stories that are estimated to be too large

- **Lab:**
  - Lab available in eLearn
  - Learn how to store estimates in Jira
  - Learn how to plan and start a sprint



Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet ☺
- **http://smu.sg/is212-reflections-2025**

